# CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)

Assistant
Teaching Professor
Lisa Yan

# Pipeline I: 5-Stage Pipeline

# Call home, we've made HW/SW contact!

High Level Language Program (e.g., C)

Compiler

Assembly Language Program (e.g., RISC-V)

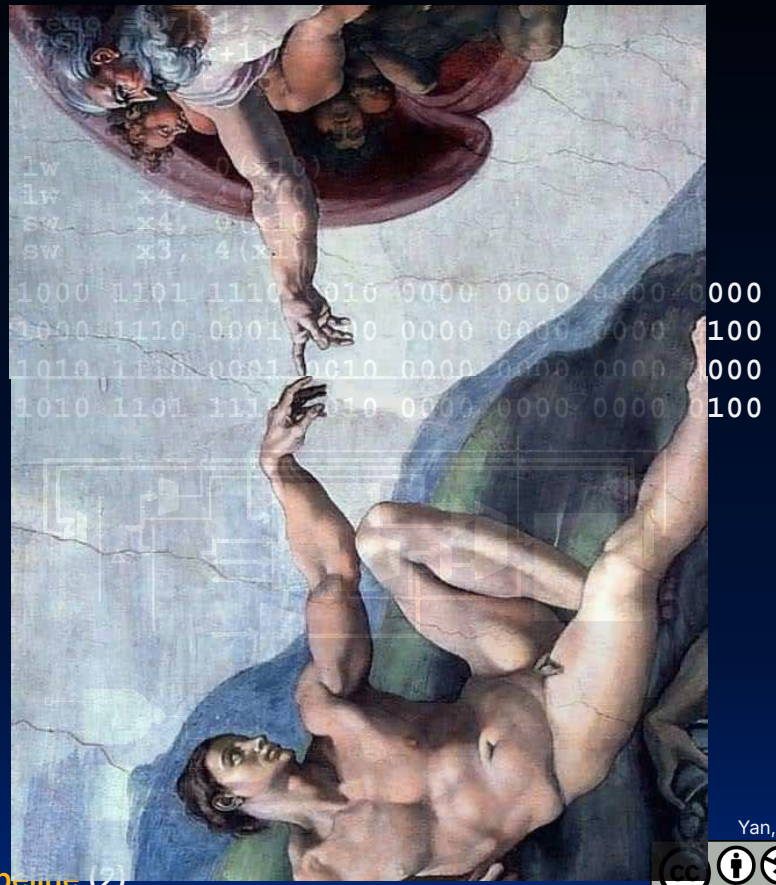
Assembler

Machine Language Program (RISC-V)

Hardware Architecture Description (e.g., block diagrams)

Architecture Implementation

Logic Circuit Description (Circuit Schematic Diagrams)



# Single-Cycle Datapath

- We have built a processor!
  - Capable of executing all RISC-V instructions in one cycle each
- Not all HW units used by all instructions
  - The **critical path changes** based on instruction.
- 5 Phases of execution
  - IF, ID, EX, MEM, WB
  - Not all instructions are active in all phases
- Controller specifies how to execute instructions
  - Implemented as ROM or logic

# Agenda

## Latency vs. Throughput

- Performance Measures
- Pipeline Registers
- Pipelined RV32I Datapath
- [at home] Pipelined RV32I Processor Details

# 6 Great Ideas in Computer Architecture

1. Abstraction (Layers of Representation/Interpretation)
2. Moore's Law
3. Principle of Locality/Memory Hierarchy
4. Parallelism
5. **Performance Measurement & Improvement**
6. Dependability via Redundancy

# Great Idea #5: Performance Measurement & Improvement

Match application to underlying hardware to exploit:

- Locality;
- Parallelism;
- Special hardware features, like specialized instructions (e.g., matrix manipulation).

Latency and Throughput, colloquially:

- "Latency": **How long** end-to-end it takes to complete a task (or program/set of tasks)
- "Throughput": How efficiently it executes tasks **once it gets going**

# Transportation Analogy

| | Sports Car  | Bus  |
|---|---|---|
| **Passenger Capacity** | 2 people | 50 people |
| **Travel Speed** | 200 mph | 50 mph |

Task: A 50-mile round trip for 100 passengers.
Which car performs "**better**"? Depends on the **performance measure**!

| | | |
|---|---|---|
| 1. Time per trip | ? | ? |
| 2. Time for 100 passengers | ? | ? |
| 3. Passengers/hour | ? | ? |

Any other metrics?

# Transportation Analogy

| | Sports Car  | Bus  |
|---|---|---|
| **Passenger Capacity** | 2 people | 50 people |
| **Travel Speed** | 200 mph | 50 mph |

Task: A 50-mile round trip for 100 passengers.
Which car performs "**better**"? Depends on the **performance measure**!

| | | |
|---|---|---|
| 1. Time per trip | **15 minutes** ✅ | 60 min |
| 2. Time for 100 passengers | 750 min (50 2-person trips) | **120 min** ✅ (2 50-person trips) |
| 3. Passengers/hour | 8 | **50** ✅ |

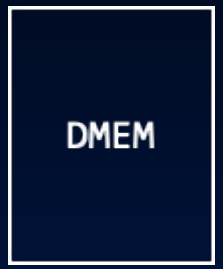
(also: energy efficiency and gas mileage, cost over car lifetime...)

# Transportation → Computers

| Transportation Analogy | Measure | Processor |
|---|---|---|
| 1. Trip time per ride | **Latency** | **Instruction latency** |
| 2. Time for 100 passengers | **Latency** | **Program execution time** (e.g., time to update display) |
| 3. Passengers/hour | **Throughput** | **Instruction throughput** (instructions/cycle) |

# Datapath: How to set clock frequency?

Assume each phase is **dominated** by major functional HW units:

| Instruction Fetch (IF) | Instruction Decode (IF) | Execute (EX) | Memory Access (MEM) | Write back to Reg (WB) |
|---|---|---|---|---|
|  |  |  |  |  |
| 200 ps | 100 ps | 200 ps | 200 ps | 100 ps |



MUX + dataW setup time (because write occurs on next cycle)

# Compute Clock Period

- The clock cycle must encompass the **longest critical path delay** incurred by any instruction.

| Instruction | IF (200ps) | ID (100ps) | EX (200ps) | MEM (200ps) | WB (100ps) | Total |
|---|---|---|---|---|---|---|
| `add` | X | X | X | | X | **600ps** |
| `beq` | X | X | X | | | **500ps** |
| `jal` | X | X | X | | | **500ps** |
| **`lw`** | X | X | X | X | X | **800ps** |
| `sw` | X | X | X | X | | **700ps** |

- Maximum Clock Frequency:
  - $f_{max}$ = **1/800 ps = 1.25 GHz**
  - However, not all instructions use all HW blocks.
- Many blocks are idle most of the time…!
  - Conversely, each phase just takes 200 ps → could clock 5GHz…??

# Agenda

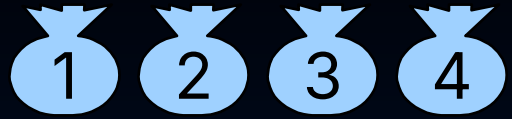
## Intro to Pipelining: Laundry

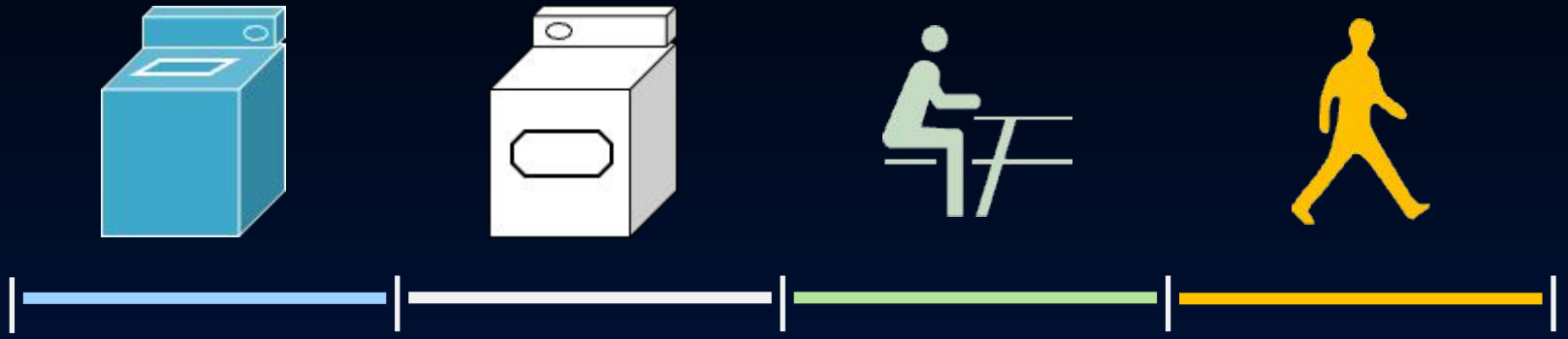
- Performance Measures
- Pipeline Registers
- Pipelined RV32I Datapath
- [at home] Pipelined RV32I Processor Details

How can we improve **instruction throughput**?

# Gotta Do Laundry

- Four customers (1, 2, 3, 4) each have one load of clothes to wash, dry, fold, and put away.



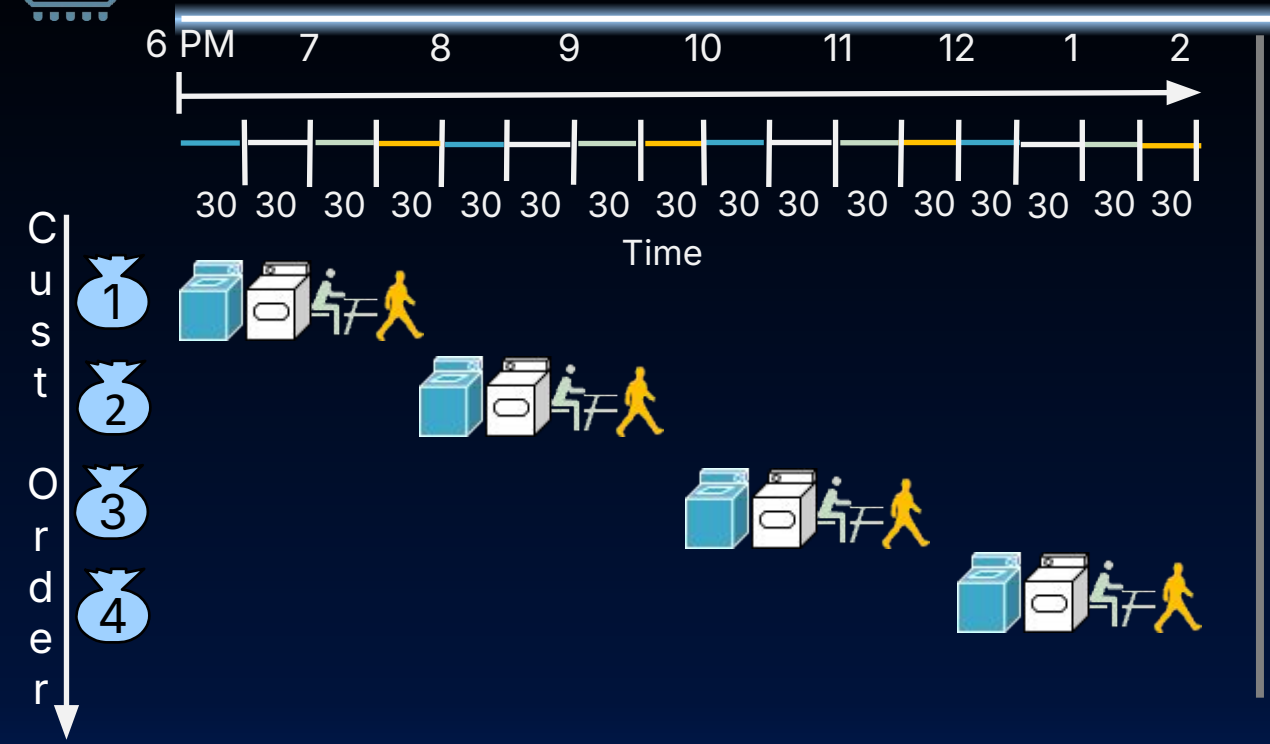
Washer Stage
30 minutes

Dryer Stage
30 minutes

“Folder” Stage
30 minutes

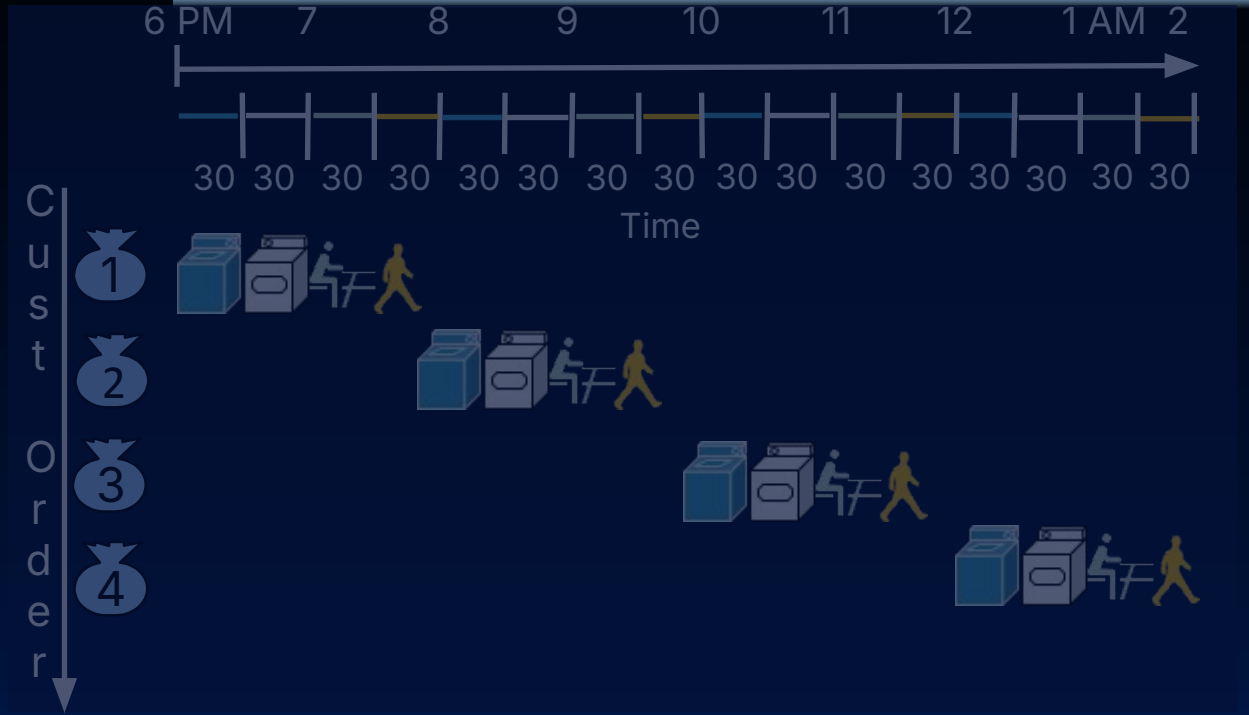
“Stasher” Stage
30 minutes
(put clothes in drawers)

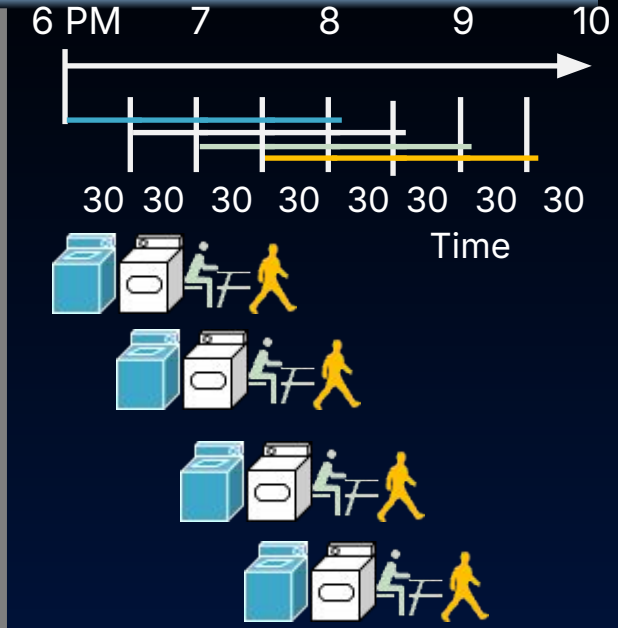
# Sequential Laundry




**Sequential** laundry takes
8 hours for 4 loads!

# Parallel, Pipelined Laundry

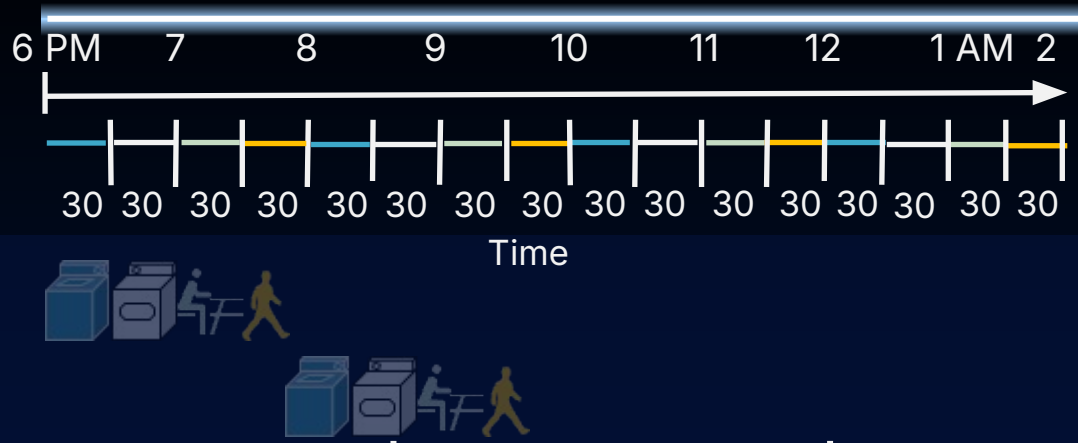


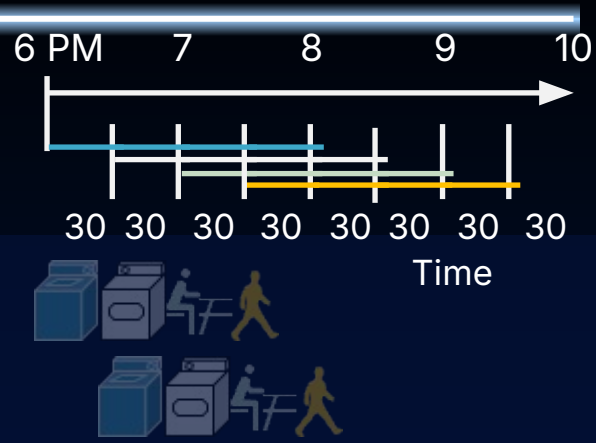




**Sequential** laundry takes
8 hours for 4 loads!

**Parallel** laundry takes
3.5 hours for 4 loads!

# Latency vs. Throughput




| | | Sequential | Pipelined |
|---|---|---|---|
| Program execution time | Time to finish all 4 loads | 8 hours | 3.5 hours |
| Instruction Latency | Time to finish a single load | 2 hours | 2 hours |
| Instruction Throughput | Avg number of loads per 30 min | 0.25 | 1 |

Pipelining increases instruction throughput, **not** instruction latency.

# Latency vs. Throughput [details]

| | | Sequential | Pipelined |
|---|---|---|---|
| Program execution time | Time to finish all **4 loads** | 8 hours | 3.5 hours |
| Instruction Latency | Time to finish a single load | 2 hours | 2 hours |
| Instruction Throughput | Avg number of loads per 30 min | 0.25 | 1 |

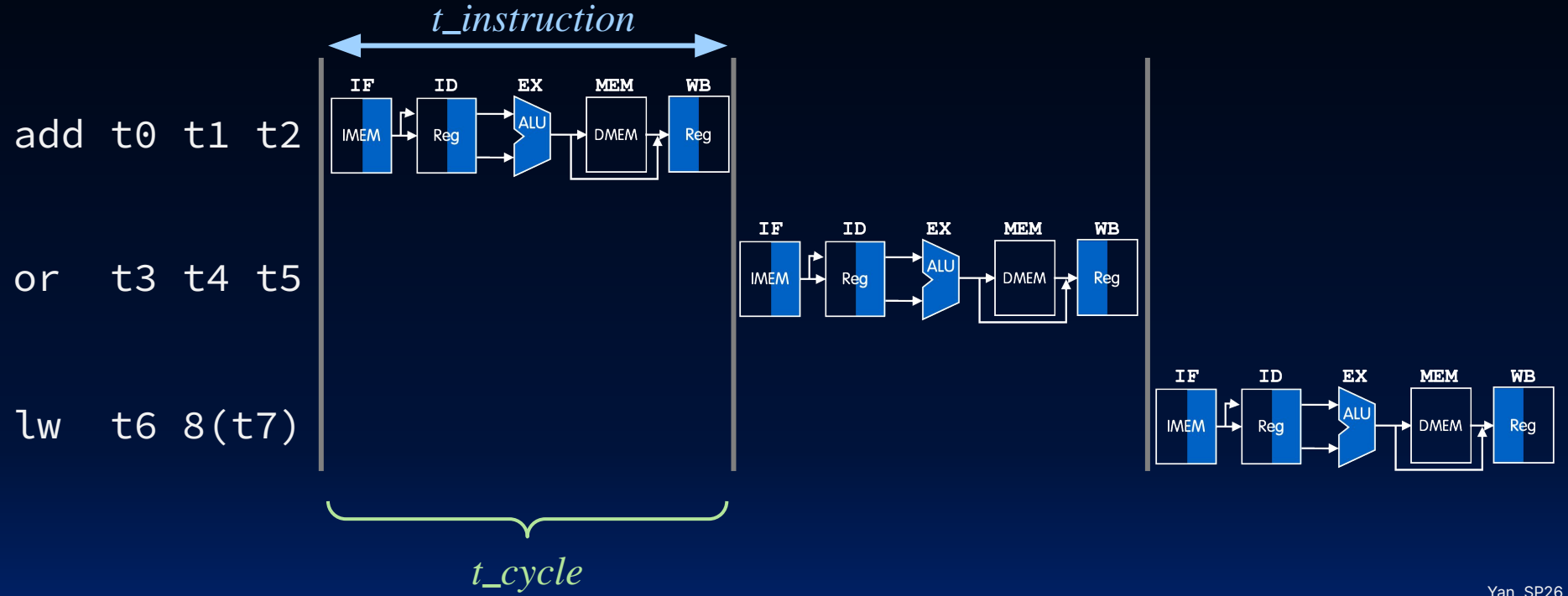
$$\frac{\text{time}}{4\ \text{loads}} = \frac{3.5}{4}\frac{\text{hours}}{\text{loads}} \cdot 2\frac{30\ \text{min}}{\text{hour}} = \frac{7}{4}\left(\frac{30\ \text{min}}{\text{loads}}\right)$$

$$\rightarrow \frac{4}{7}\frac{\text{loads}}{30\ \text{min}} \approx 0.57\frac{\text{loads}}{30\ \text{min}}$$

But not on average... :-) Instead, once the pipeline is fully "filled," exactly **1 load** (customer) will finish each 30 minutes.

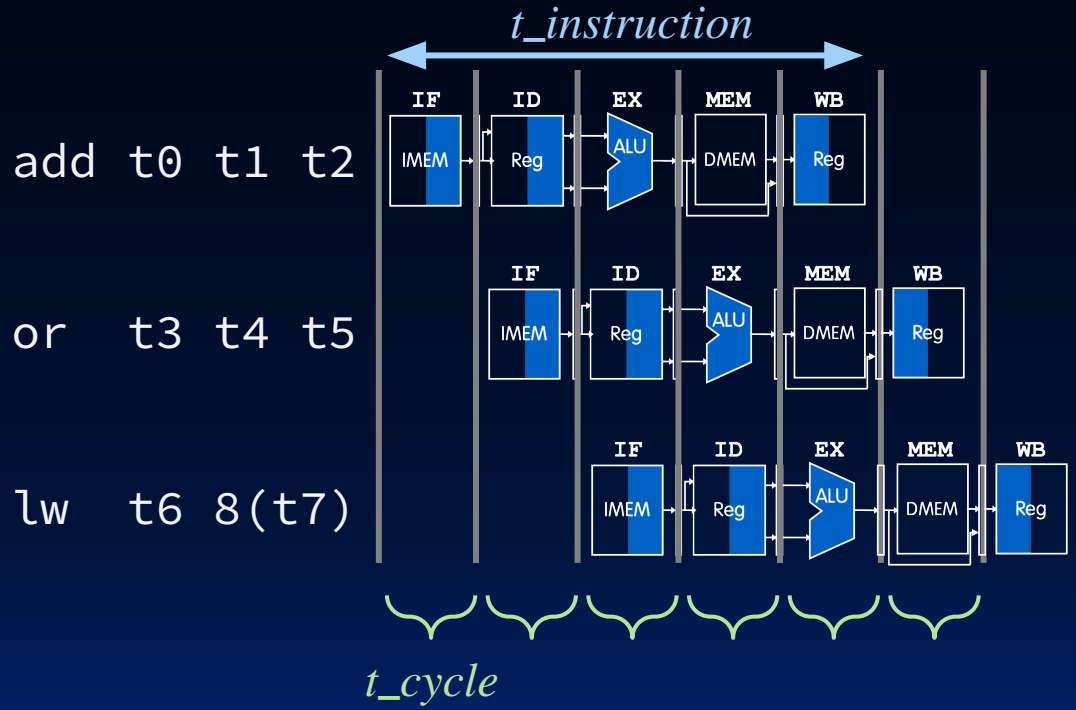
# "Sequential" RISC-V Datapath

In a **single-cycle CPU**, only one instruction can access any resources in one clock cycle, in sequence.



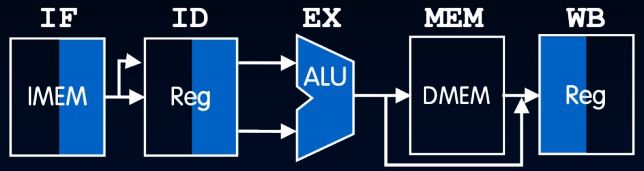
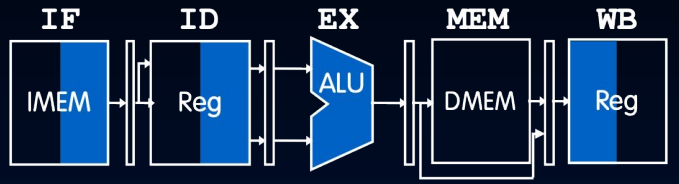
# Pipelined RISC-V Datapath

In a **pipelined CPU**, multiple instructions access resources in one clock cycle.




**Pipelined datapath**
(how??? up next)

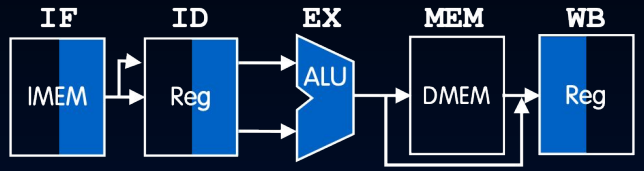
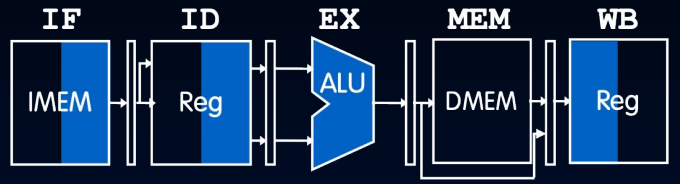
# Performance Comparison

| | Single-Cycle Datapath | Pipelined Datapath |
|---|---|---|
| High-level diagram |   |   |
| Stage Timing | $t\_stage$ = 200, 100, 200, 200, 100 ps (Reg stages ID, WB only 100 ps) | $t\_stage$= 200 ps All stages same length |
| Instruction Latency | $t\_instruction$ = 800 ps | $t\_instruction$=5 $t\_cycle$ = 1000 ps |

- The pipelined CPU still uses one clock for all stages!
- Clock cycle time is limited by the slower stages.

# Performance Comparison

throughput gain x4

| | Single-Cycle Datapath | Pipelined Datapath |
|---|---|---|
| High-level diagram |   |   |
| Stage Timing | $t\_stage$ = 200, 100, 200, 200, 100 ps (Reg stages ID, WB only 100 ps) | $t\_stage$= 200 ps All stages same length |
| Instruction Latency | $t\_instruction$ = 800 ps | $t\_instruction$=5 $t\_cycle$ = 1000 ps |
| Clock frequency | 1/$t\_instruction$ = 1/800 ps = 1.25 GHz | 1/$t\_cycle$ = 1/200 ps = 5 GHz |

# Agenda

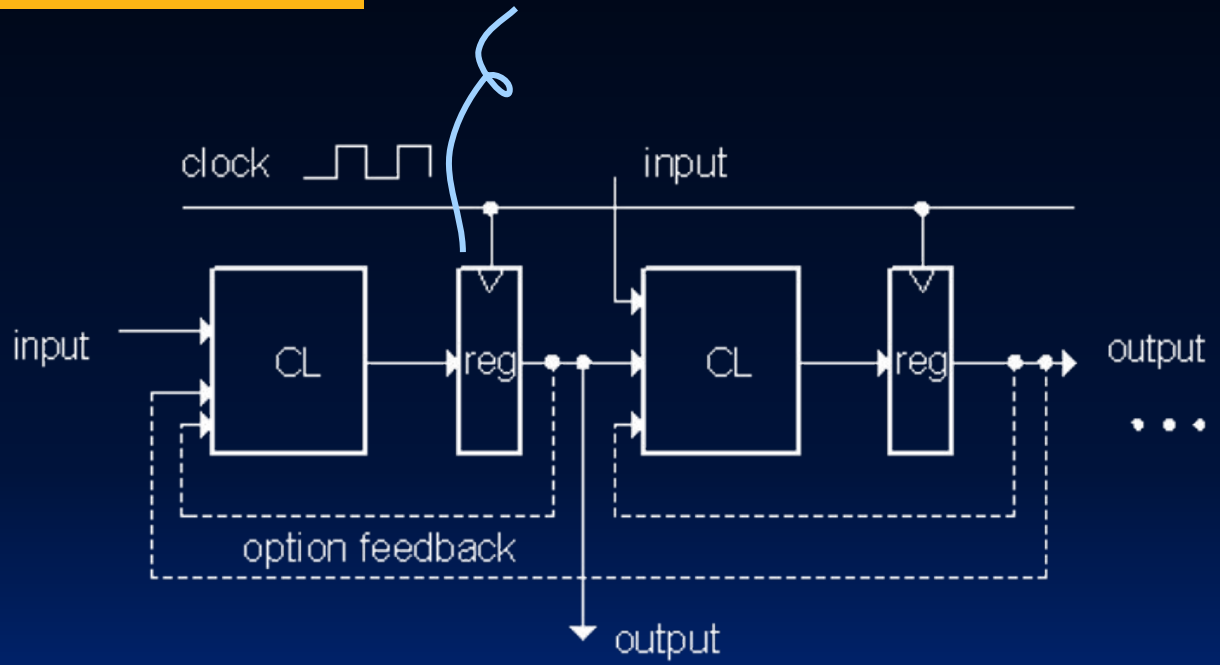
## Pipelining: Motivator

- Performance Measures
- Pipeline Registers
- Pipelined RV32I Datapath
- [at home] Pipelined RV32I Processor Details

# General Model for Synchronous Systems, again

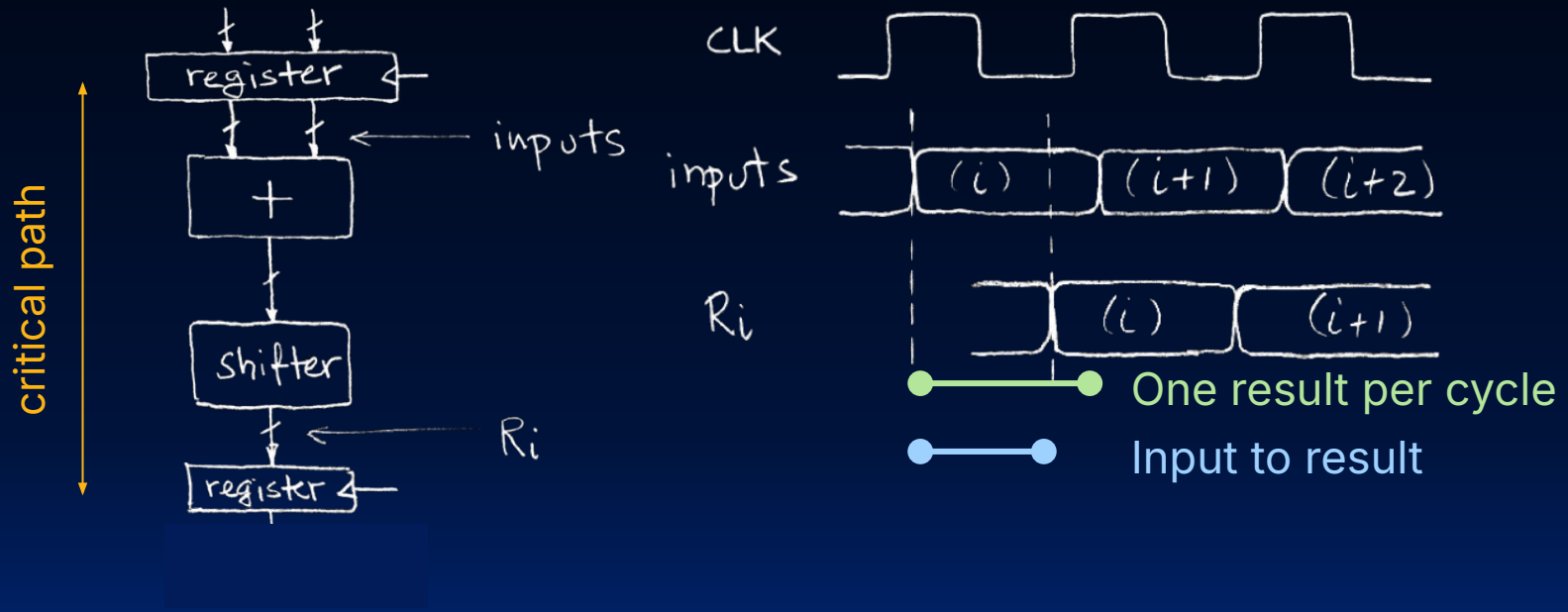
The idea: registers can **improve instruction throughput**.

A **pipeline** register



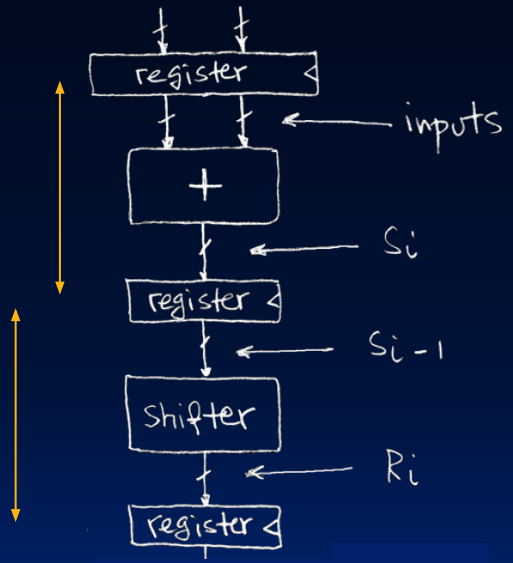
# Non-Pipelined Adder/Shifter Circuit

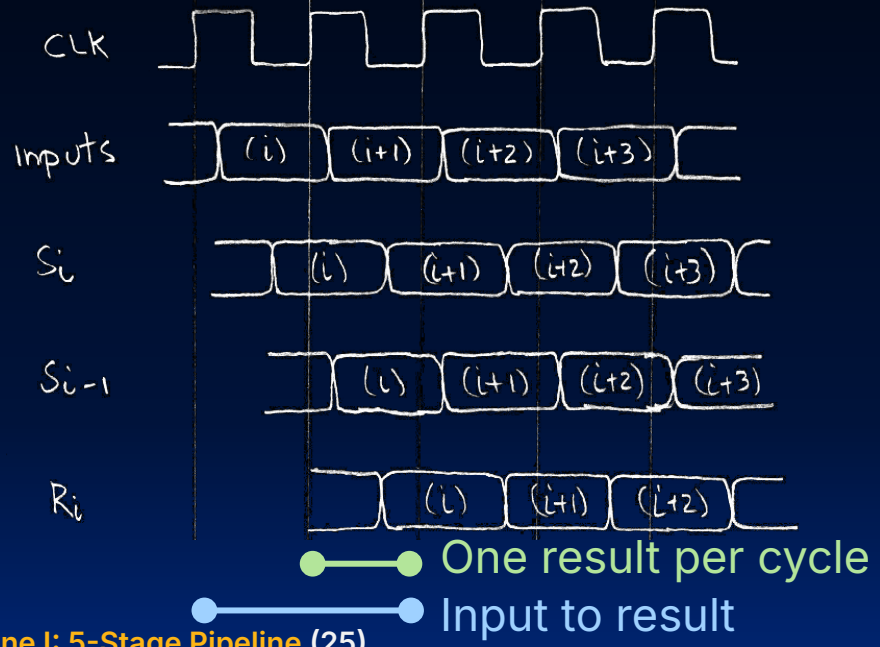
- **Without a pipeline**, delay of 1 clock cycle to compute output.
  - Clock period is limited by the propagation delay of adder + shifter.



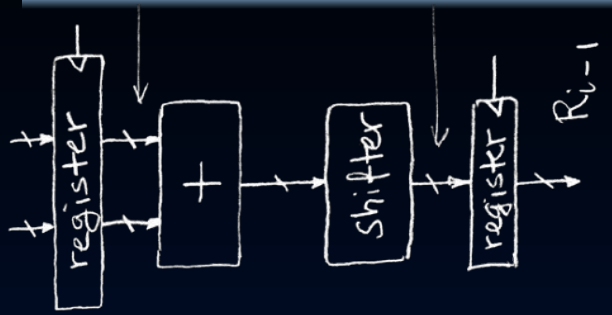
# Pipelined Adder/Shifter Circuit

- With **pipeline register**: Two stages: shifter and adder, occurring each cycle
  - Critical path delay = max{shifter stage, adder stage}
  - ✅ Higher **throughput**: More outputs/second, but…
  - ⚠ Higher **latency**: Each individual result ***may*** take longer.



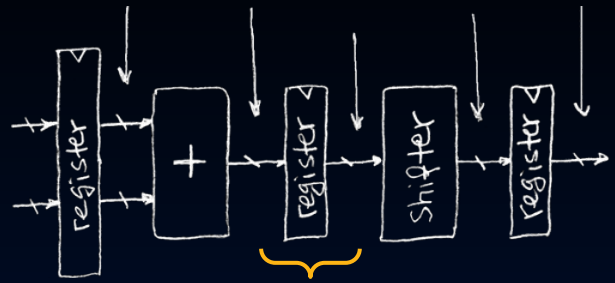



# Pipeline: Clock Frequency, Latency




Critical path delay = tclk_q + tAdd
+ tShift + tSetup

latency = tclk_q + tAdd + tShift + tSetup




**pipelined register**

critical path delay = **max**{
tclk_q + tAdd + tSetup,
tclk_q + tShift + tSetup
}

latency = tclk_q + tAdd + tSetup
+ tclk_q + tShift + tSetup

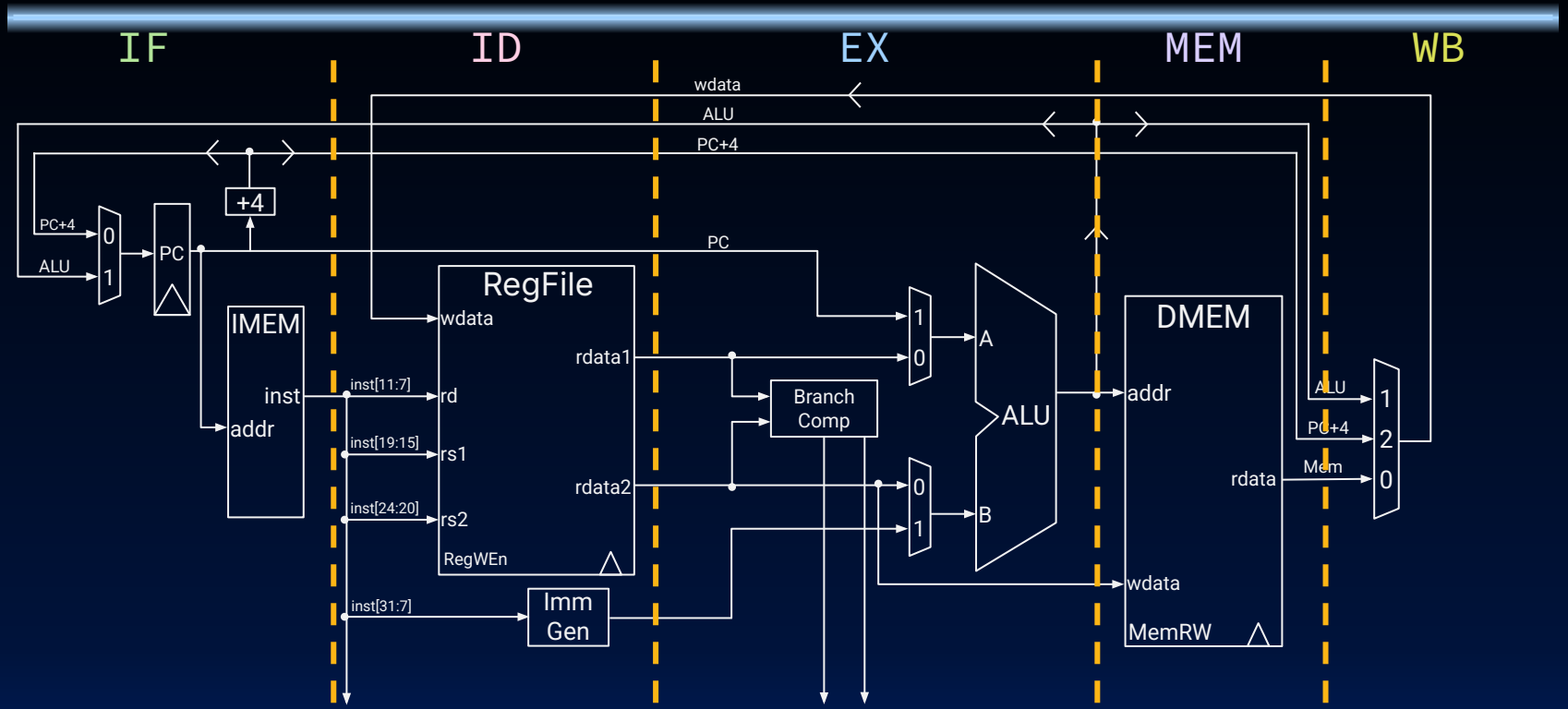
$$\frac{\text{time}}{\text{program}} = \frac{\text{instructions}}{\text{program}} \cdot \frac{\text{cycles}}{\text{instructions}} \cdot \frac{\text{time}}{\text{cycle}}$$

# Agenda

## Pipelined RV32I Datapath

- Performance Measures
- Pipeline Registers
- Pipelined RV32I Datapath
- [at home] Pipelined RV32I Processor Details

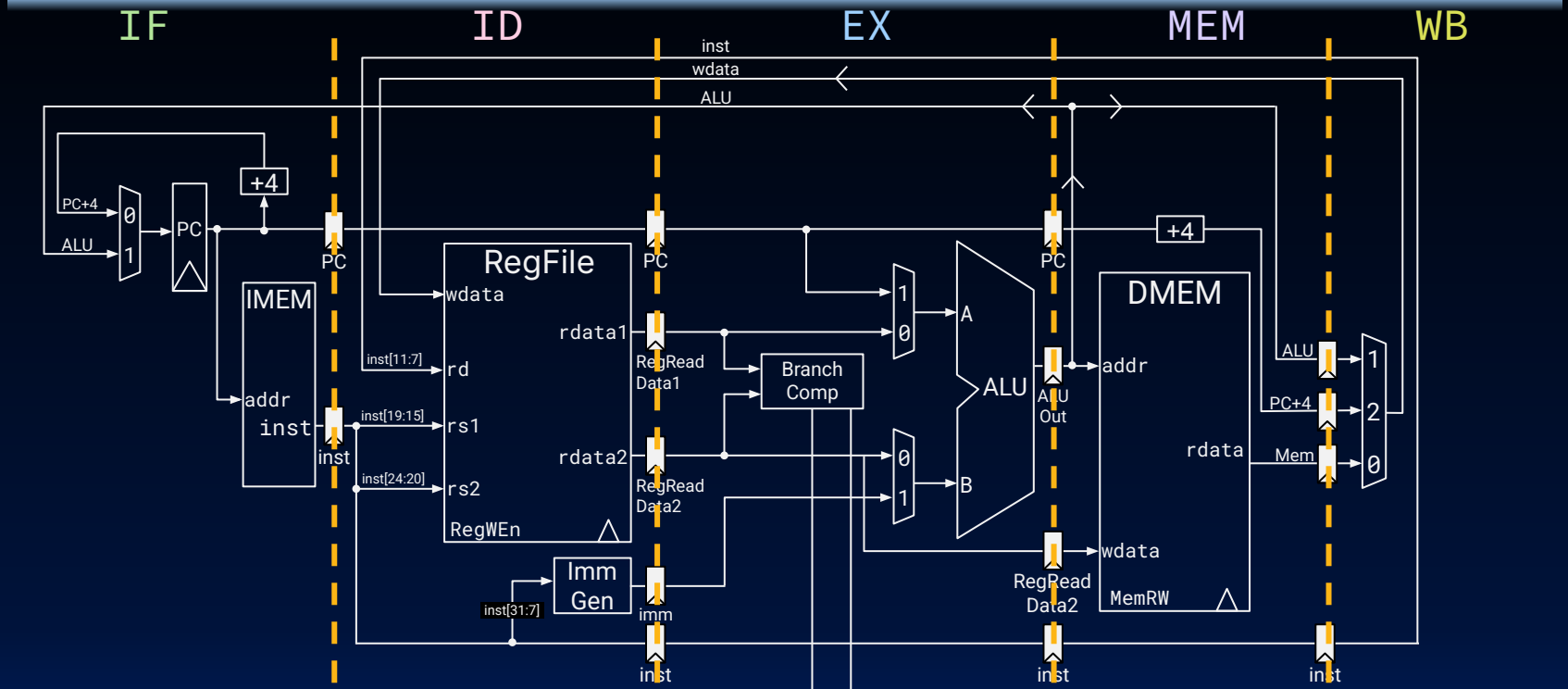
# 5-Stage Datapath for RV32I?




A pipelined datapath needs to "separate" the five stages of the RV32I datapath.

- Each stage needs to process data from a different instruction.

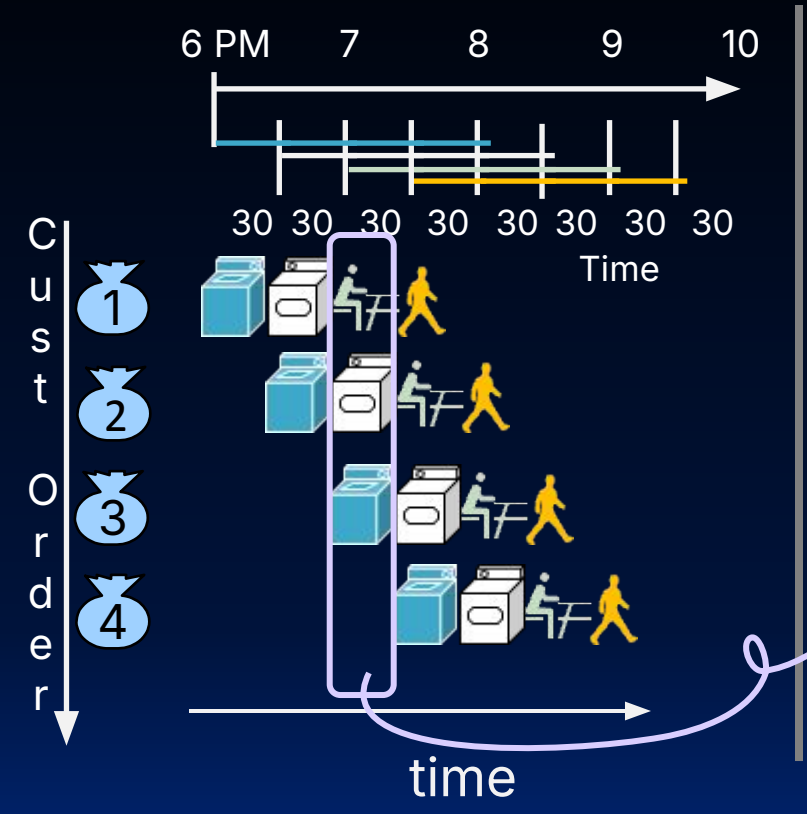
# Pipelined Registers Separate the 5 Stages



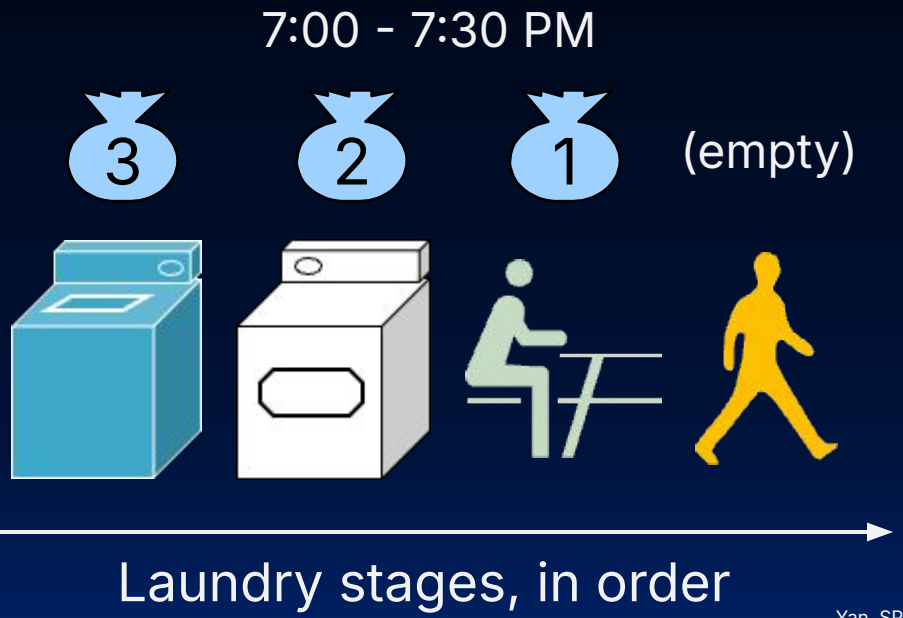

@ rising clock edge: pipeline registers carry data/control signals to next stage.

Read more details at home (last section)

# Note: Two Views

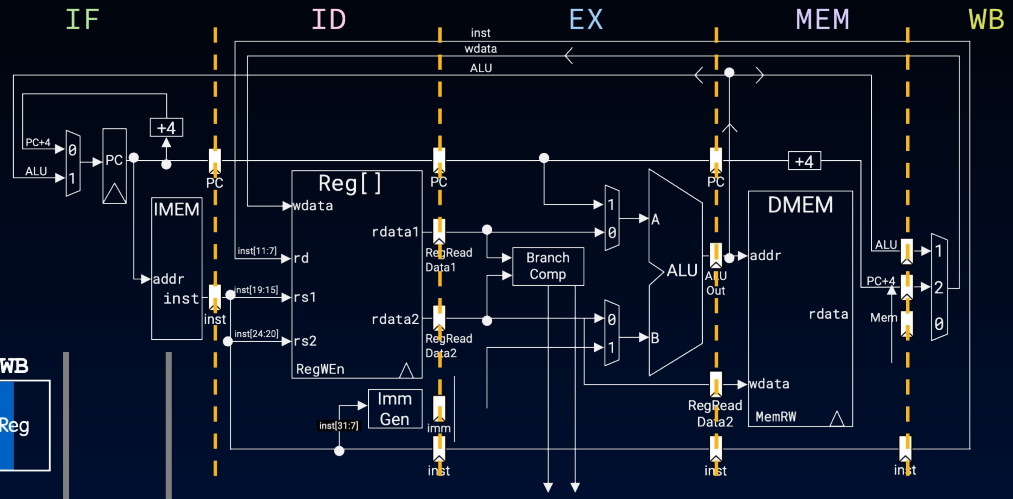



Earlier customers are in **later** pipeline stages (they are closer to finishing their laundry load).

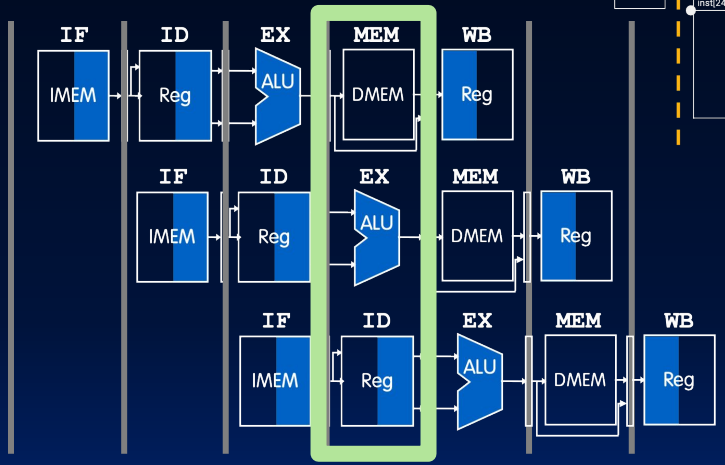


# Simultaneously executing instructions

In the **provided clock cycle**, how are all simultaneously executing instructions using the pipelined stages of the datapath?

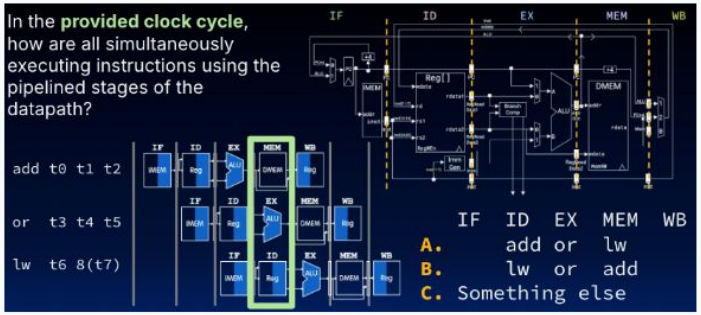



```
add  t0 t1 t2
or   t3 t4 t5
lw   t6 8(t7)
```




```
     IF   ID   EX   MEM  WB
A.        add  or   lw
B.        lw   or   add
C.  Something else
```

# L27: In the provided clock cycle, how are all simultaneously executing instructions using the pipelined stages of the datapath?




A

0%

B

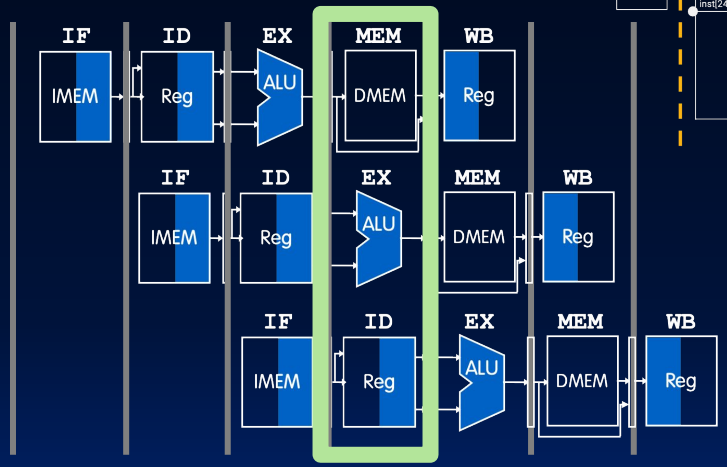
0%

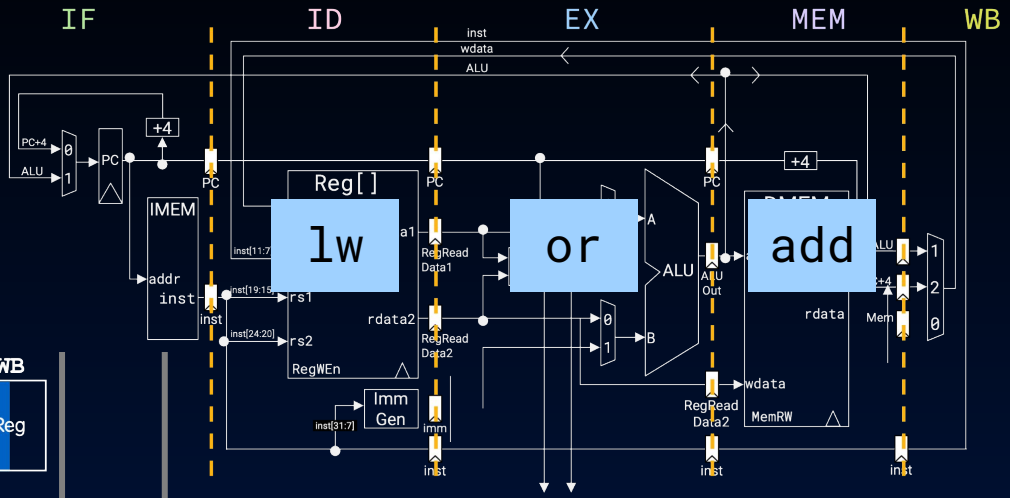
C. Something else

0%

# Simultaneously executing instructions

```
add  t0 t1 t2
or   t3 t4 t5
lw   t6 8(t7)
```




B.

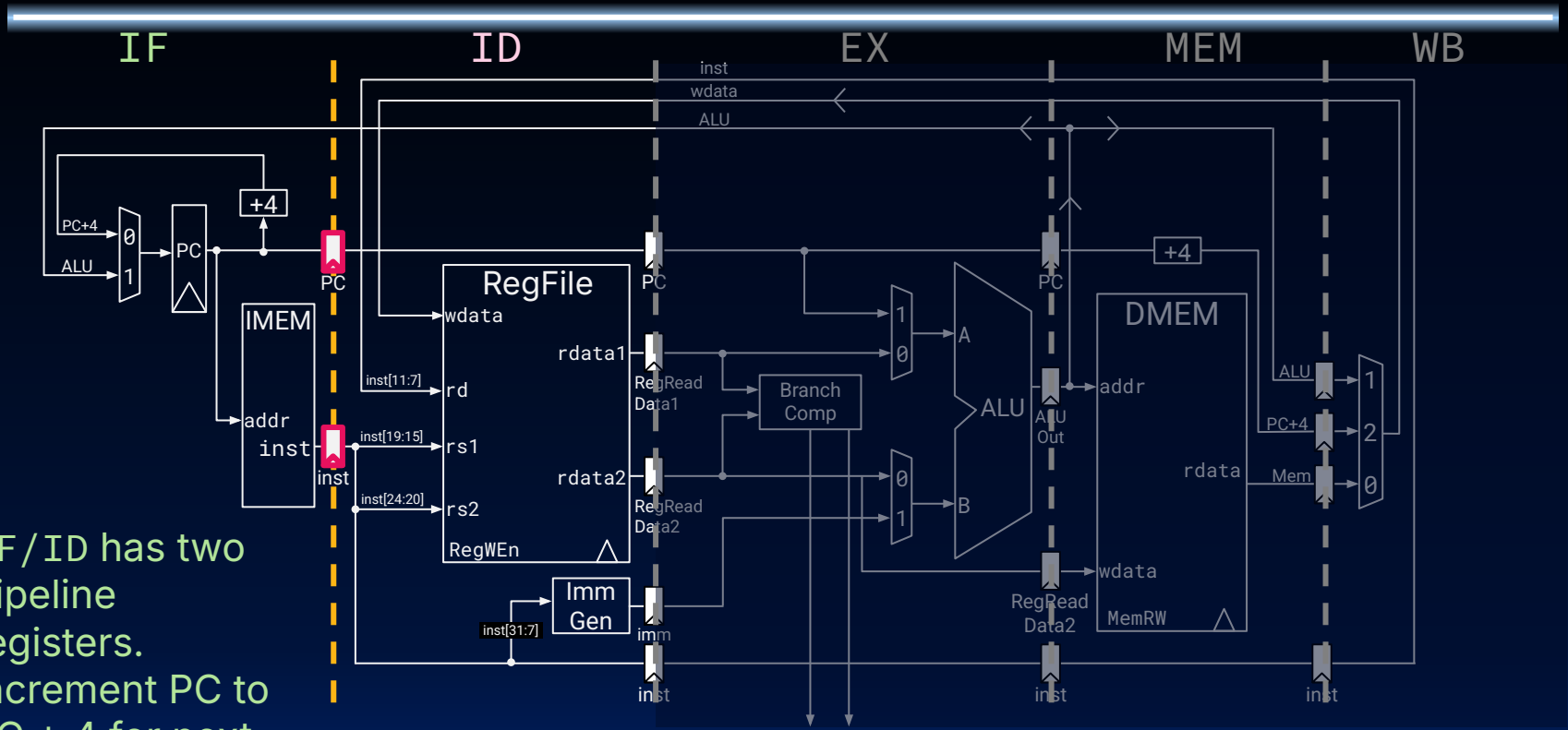
# Agenda

[read at home]

## [at home] Pipelined RV32I Processor Details

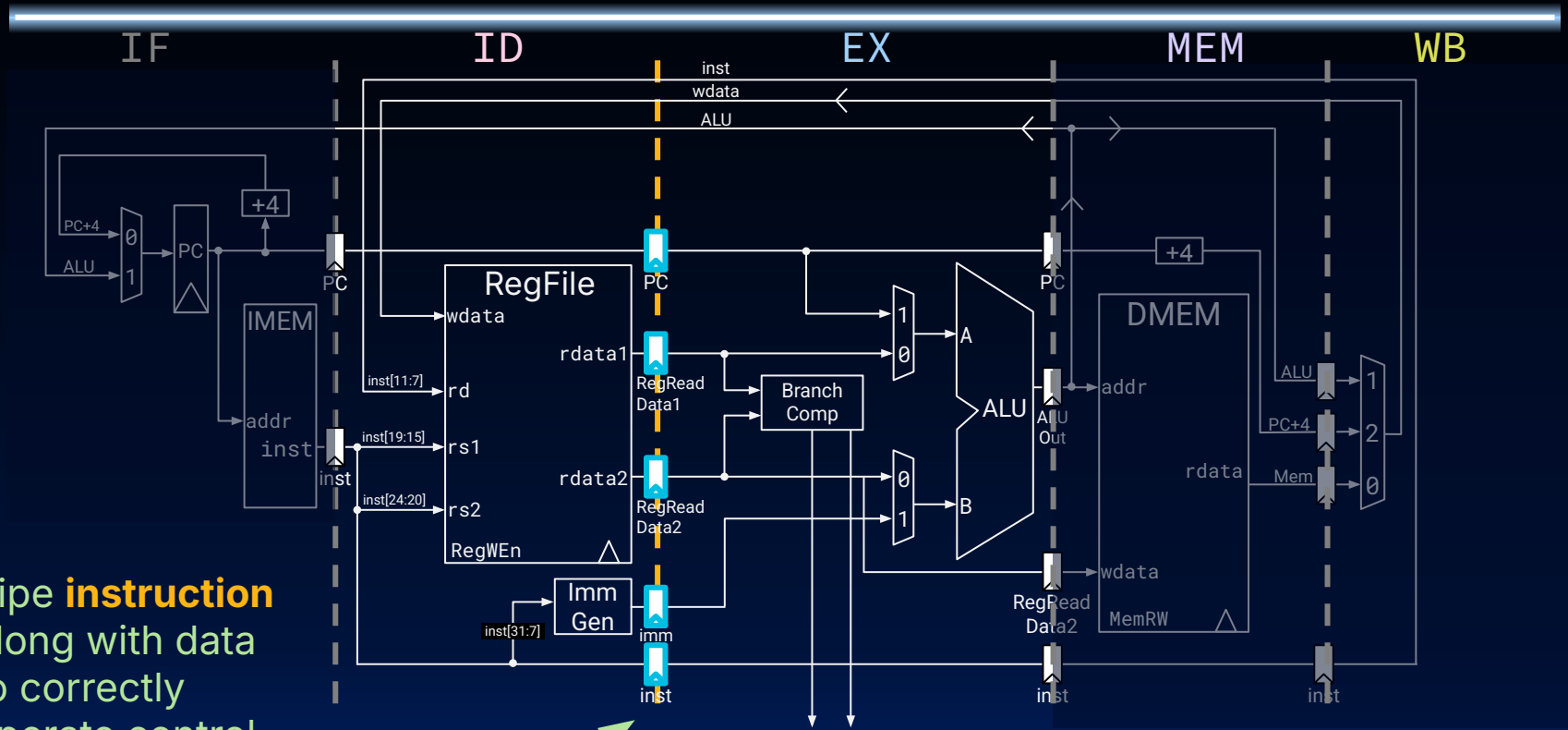
- Performance Measures
- Pipeline Registers
- Pipelined RV32I Datapath
- [at home] Pipelined RV32I Processor Details

# Datapath Pipeline Registers (1/4): IF/ID



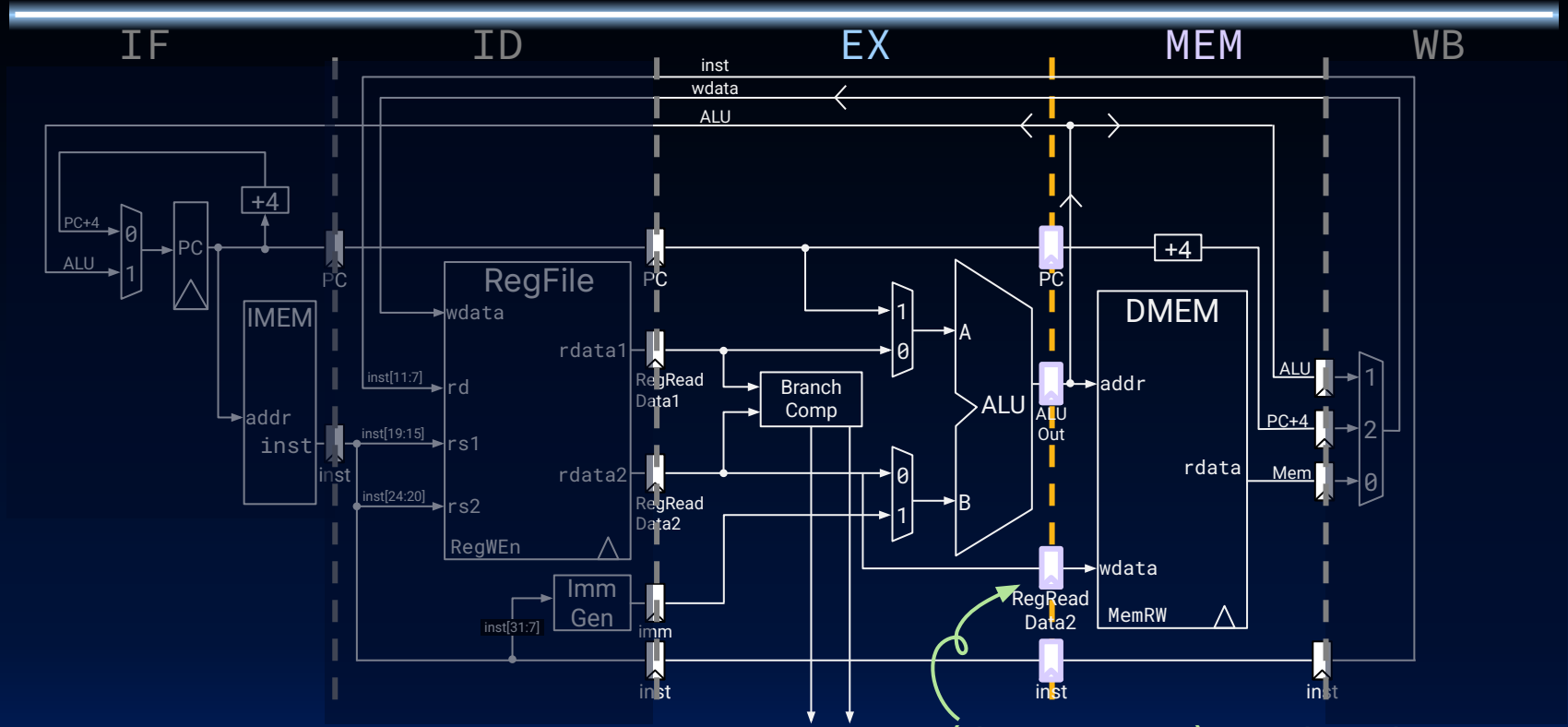

- `IF/ID` has two pipeline registers.
- Increment PC to PC + 4 for next cycle's `IF` stage.

# Datapath Pipeline Registers (2/4): ID/EX

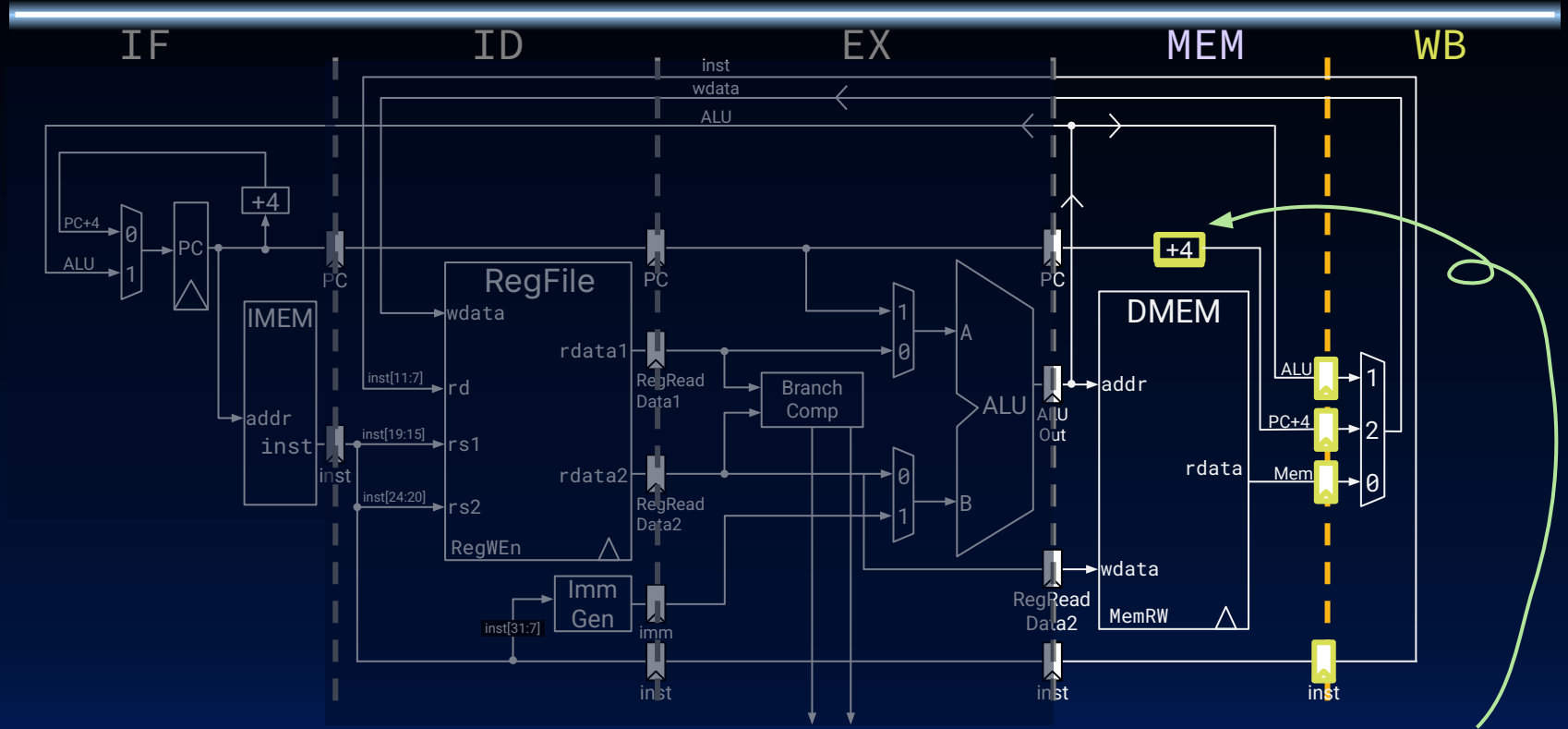



- Pipe **instruction** along with data to correctly operate control in each stage.

# Datapath Pipeline Registers (3/4): EX/MEM

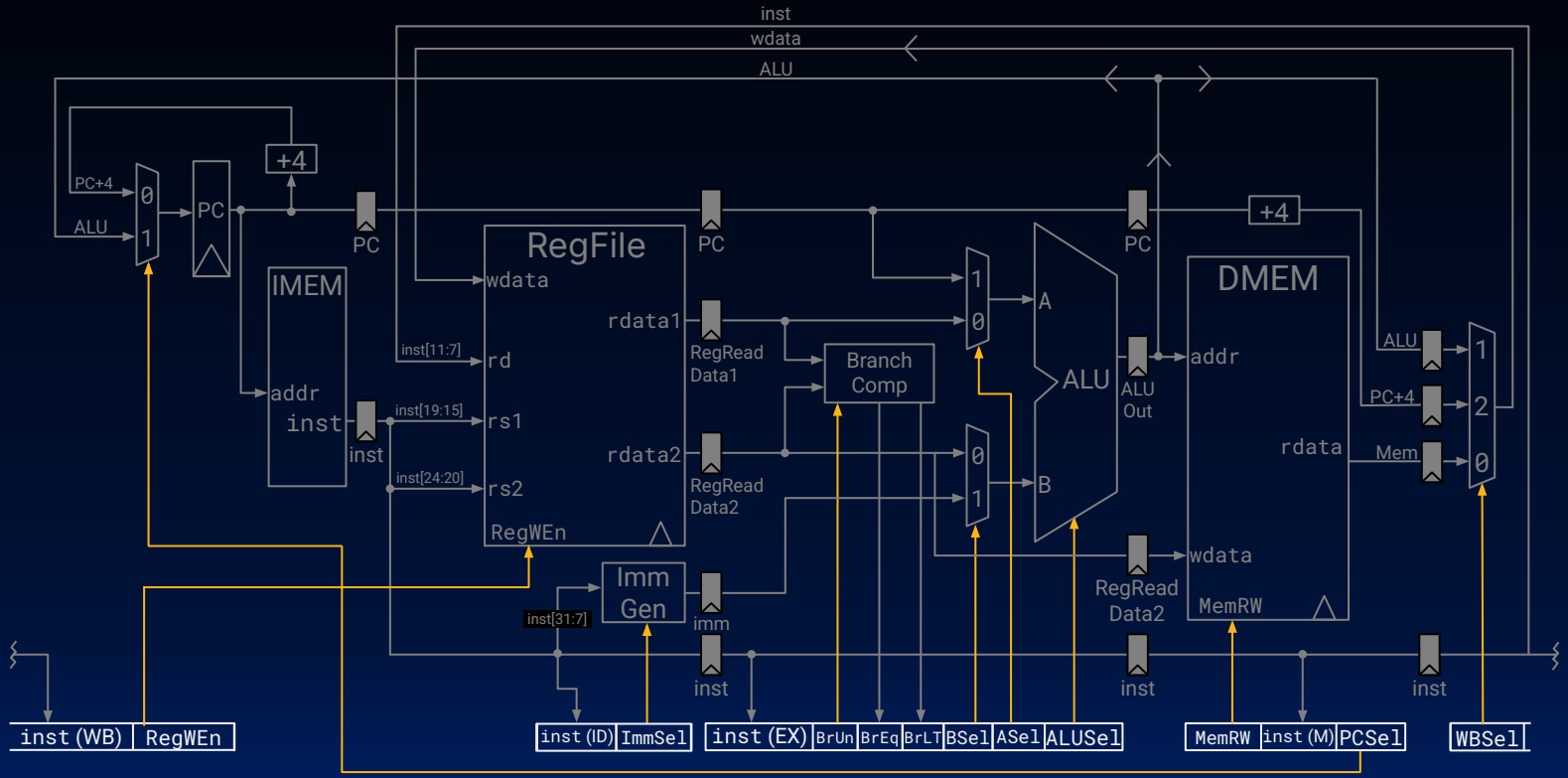


# Datapath Pipeline Registers (4/4): MEM/WB



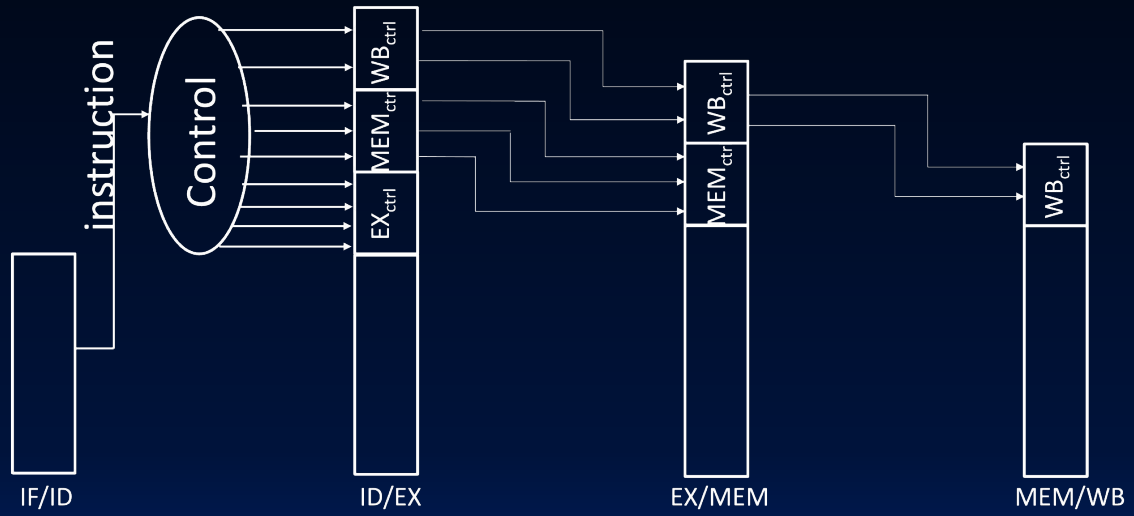

To avoid sending both PC, PC+4 down pipeline, recalculate PC+4 here.

# Control Is Also Pipelined (1/2)



# Control Is Also Pipelined (2/2)

- Control signals are derived from the instruction.
  - Control information for later stages is also stored in **pipeline registers**.
- One strategy:
  - Compute as many control signals as possible during instruction decode (`ID`).
  - Then, pipeline control "words" between stages.



# 5-Stage Datapath for RV32I: Reference